

PhD thesis summary

Deep Inference for Graphical Theorem Proving

Pablo Donato

Introduction

Proof assistants are software tools that rigorously verify mathematical reasoning. They can be general-purpose (like Coq [1], Lean [2], and Isabelle [3]) or more specialized (like EasyCrypt [4] for the verification of cryptographic protocols). While they provide a level of precision that ensures no errors can occur in an argument or calculation, they remain difficult to use.

My research introduces a new paradigm for constructing formal proofs through actions performed in a graphical user interface (GUI), aiming to make proof development in these software more intuitive and accessible to a broader audience. Dubbed Proof-by-Action (PbA), this paradigm is based on principles of direct manipulation with a pointing device such as a mouse or finger on a touchscreen, contrasting with current interfaces based on textual scripting languages. It combines and extends existing interaction techniques from the literature like Proof-by-Pointing (PbP) [5] and Proof-by-Linking (PbL) [6], grounding them in a recent branch of structural proof theory called “deep inference” [7].

Key Contributions and Original Results

Symbolic Manipulations

The first part of this thesis develops the PbA paradigm in the context of traditional representations of logical statements, introducing several techniques for direct manipulation of symbolic formulas.

- We explore PbP and PbL and demonstrate how logical connectives, quantifiers, and equations can be manipulated through clicks and drag-and-drop (DnD) actions in a prototype GUI called Actema.
- A key result is the firm foundation of DnD actions within deep inference proof theory, enabling productive user interactions with the help of a deterministic algorithm that only suggests useful reasoning steps.
- Advanced PbA techniques are introduced for pervasive mathematical reasoning tasks such as using definitions, reasoning by induction, and performing simple calculations. Those features are illustrated through case studies on basic logical problems given in undergraduate computer science and mathematics curricula.
- The previous results culminate in the implementation of coq-actema, a plugin integrating Actema as an interactive proof editor for the Coq proof assistant. It details the system architecture, protocols, and compilation strategies for translating graphical actions into formal proofs expressed in Coq’s language.

Diagrammatic Manipulations

The second part of the thesis explores deep inference proof systems that enrich the structure of logical statements, offering both textual and diagrammatic notations that ease gestural manipulation.

- We introduce *bubble calculi*, a way to represent logical statements visually as nested bubbles, making it easier to track how information flows in arguments. This builds on earlier methods in the literature, but allows for more efficient reuse of shared assumptions and conclusions.
- We give a modern account of C. S. Peirce’s *existential graphs* (EGs), a diagrammatic notation for logical reasoning that predates the development of formal logic. We show how EGs remove the need for logical symbols, by mathematically proving properties of the system that justify it as a viable alternative to traditional proof calculi.
- We introduce the *flower calculus*, an extension of EGs based on a metaphorical representation of logical statements as flowers. It captures *constructive* reasoning in addition to classical logic, paving the way for applications to programming and proof assistants.
- We finally go back to practical experimentation by designing and implementing the *Flower Prover*, another GUI prototype for interactive proof building based on the flower calculus. An innovative feature of the Flower Prover is that it works well on modern mobile devices, thanks to its responsive layout and first-class support for touch interactions.

Broader Impact and Applications

Academic Impact

Some results of this thesis have been published in proceedings of top-tier conferences in theoretical computer science:

- the theory of DnD actions underlying the Actema system, at the international conference on Certified Programs and Proofs (CPP 2022) with my PhD supervisors as co-authors [8];
- the flower calculus, at the international conference on Formal Structures for Computation and Deduction (FSCD 2024) as a completely personal contribution [9].

According to Google Scholar, the CPP paper has been cited 20 times, and the more recent FSCD paper 3 times. In particular, one paper at CPP 2024 by Robert Krebbers (a top researcher in the field who was chair of CPP 2022) gives a thorough study of the DnD technique in Actema, fixing one of its limitations and applying it to automated theorem proving in the Iris framework for program verification [10].

These results and others have also been presented at various international workshops, working groups and seminars, as indicated in my CV.

Practical Applications

Beyond academic contributions, I envision three main areas of application where these results could have a significant impact:

Mathematics Education

The most immediate application of the PbA paradigm is in the teaching of formal logic and proof writing. By providing a more intuitive interface for constructing proofs, students can

focus on the logical structure of their arguments rather than the syntactic details of a formal language. This could lead to more engaging and effective learning experiences, especially for students who struggle with traditional proof methods. The Actema system and the Flower Prover could be used as teaching tools in undergraduate courses on discrete mathematics, logic, and theoretical computer science, helping students develop their proof-writing skills in a more interactive and visual way.

Visual Programming

Through the Curry-Howard correspondence between proofs and programs (as exemplified by Coq and Lean), the PbA paradigm could also be applied to the design of visual programming languages, where users construct programs by manipulating graphical elements on a canvas. This approach has been popularized by tools like Scratch [11] and Blockly, which are used to teach programming concepts to children and beginners. By extending the PbA techniques developed for logical reasoning to the domain of programming, we could create a new generation of visual programming environments that are more expressive and powerful, allowing users to build complex but *correct-by-construction* algorithms through simple gestural interactions. This could have a significant impact on the democratization of programming, making it more accessible to a wider audience and enabling new forms of creative expression through code.

Artificial Intelligence

The graphical proof systems introduced here could influence knowledge representation in AI systems, facilitating the development of more human-like reasoning engines. The use of graphical notations for knowledge representation has a long history dating back to ancient greek philosophy, with a modern resurgence in AI in the form of various kinds of *semantic networks* [12]. Recent developments in *knowledge graphs* for the Semantic Web and *graph neural networks* for machine learning [13] highlight the importance of graphical representations for capturing complex relationships between entities and concepts.

The deep inference techniques developed in this thesis could provide a foundation to build more expressive and efficient knowledge graphs, by restricting the set of permitted graph transformations to *logically meaningful* ones (i.e. inference rules). Some research in this direction has already been done by Sowa with his *conceptual graphs*, which are also inspired by Peirce’s EGs [14].

They could also enhance current generative AI systems like LLMs (Large Language Models) that have a known tendency to hallucinate, by allowing users to logically constrain the space of “interesting” outputs through prompts expressed in an intuitive graphical language. This falls in line with the current trend of *neuro-symbolic* architectures based on automated ([15], [16]) and interactive theorem provers ([17], [18]), but with an original emphasis on *diagrammatic* in addition to symbolic reasoning.

Conclusion

My doctoral work advances the state-of-the-art in proof assistant technology by proposing new graphical languages, interfaces and algorithms for formal proof construction, that reduce the friction between human and computer representations of logical reasoning. It bridges symbolic and diagrammatic notations and contributes to both abstract proof theory and practical proof engineering, fostering new interdisciplinary approaches to interactive theorem proving with promising applications to maths education, visual programming and artificial intelligence.

Bibliography

- [1] The Coq Development Team, “The Coq Proof Assistant.” [Online]. Available: <https://doi.org/10.5281/zenodo.7313584>
- [2] L. d. Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, A. Platzer and G. Sutcliffe, Eds., Cham: Springer International Publishing, 2021, pp. 625–635.
- [3] T. Nipkow, L. C. Paulson, and M. Wenzel, *Isabelle/HOL: a proof assistant for higher-order logic*, vol. 2283. Springer Science & Business Media, 2002.
- [4] G. Barthe, F. Dupressoir, B. Grégoire, C. Kunz, B. Schmidt, and P.-Y. Strub, “EasyCrypt: A Tutorial,” in *Foundations of Security Analysis and Design VII: FOSAD 2012/2013 Tutorial Lectures*, A. Aldini, J. Lopez, and F. Martinelli, Eds., Cham: Springer International Publishing, 2014, pp. 146–166. doi: 10.1007/978-3-319-10082-1_6.
- [5] Y. Bertot, G. Kahn, and L. Théry, “Proof by pointing,” *Theoretical Aspects of Computer Software*, vol. 789. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 141–160, 1994. doi: 10.1007/3-540-57887-0_94.
- [6] K. Chaudhuri, “Subformula Linking as an Interaction Method,” *Interactive Theorem Proving*, vol. 7998. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 386–401, 2013. doi: 10.1007/978-3-642-39634-2_28.
- [7] A. Guglielmi, “Deep Inference.” Accessed: Feb. 04, 2024. [Online]. Available: <https://people.bath.ac.uk/ag248/p/DI.pdf>
- [8] P. Donato, P.-Y. Strub, and B. Werner, “A drag-and-drop proof tactic,” in *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2022. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 197–209. doi: 10.1145/3497775.3503692.
- [9] P. Donato, “The Flower Calculus,” in *9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024)*, J. Rehof, Ed., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 299. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 1–24. doi: 10.4230/LIPIcs.FSCD.2024.5.
- [10] I. Mulder and R. Krebbers, “Unification for Subformula Linking under Quantifiers,” in *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2024. New York, NY, USA: Association for Computing Machinery, Jan. 2024, pp. 75–88. doi: 10.1145/3636501.3636950.
- [11] J. Maloney, M. Resnick, N. Rusk, B. Silverman, and E. Eastmond, “The Scratch Programming Language and Environment,” *ACM Trans. Comput. Educ.*, vol. 10, no. 4, Nov. 2010, doi: 10.1145/1868358.1868363.
- [12] J. F. Sowa, “Semantic Networks,” 1992. Accessed: Feb. 04, 2024. [Online]. Available: <http://www.jfsowa.com/pubs/semnet.htm>
- [13] Z. Ye, Y. J. Kumar, G. O. Sing, F. Song, and J. Wang, “A Comprehensive Survey of Graph Neural Networks for Knowledge Graphs,” *IEEE Access*, vol. 10, no. , pp. 75729–75741, 2022, doi: 10.1109/ACCESS.2022.3191784.

- [14] J. Sowa, “Conceptual Graphs for a Data Base Interface,” *IBM Journal of Research and Development*, vol. 20, p. 336, Jul. 1976, doi: 10.1147/rd.204.0336.
- [15] T. Olausson *et al.*, “LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2023, pp. 5153–5176. doi: 10.18653/v1/2023.emnlp-main.313.
- [16] L. McGinness and P. Baumgartner, “Automated Theorem Provers Help Improve Large Language Model Reasoning,” in *EPiC Series in Computing*, EasyChair, pp. 51–31. doi: 10.29007/2n9m.
- [17] E. First, A. Jiang, and K. Yang, “NeurIPS Tutorial on Machine Learning for Theorem Proving.” 2023.
- [18] Z. Li *et al.*, “A Survey on Deep Learning for Theorem Proving.” [Online]. Available: <https://arxiv.org/abs/2404.09939>

List of 4 experts

- Herman Geuvers (herman@cs.ru.nl)
- Tarmo Uustalu (tarmo@cs.ioc.ee)
- Dale Miller (dale.miller@inria.fr)
- Laurent Lafforgue (laurent.lafforgue@huawei.com)